YOLO vs. CNN: Deep Learning Approaches for ASL Alphabet Classification

Anirudh Jayan[1], Abhinav Variyath[2],
S Tara Samiksha[3], Sarvesh Ram Kumar[4],
Aravind S Harilal[5]

[1,2,3,4,5]Amrita School of Artificial Intelligence,
Coimbatore, Amrita Vishwa Vidhyapeetham, India

[1]cb.sc.u4aie24007@cb.students.amrita.edu, [2]cb.sc.u4aie24002@cb.students.amrita.edu,
[3]cb.sc.u4aie24045@cb.students.amrita.edu, [4]cb.sc.u4aie24048@cb.students.amrita.edu,
[5]cb.sc.u4aie24008@cb.students.amrita.edu

***Abstract*—American Sign Language (ASL) is a series of symbols used by deaf and mute individuals to communicate their thoughts and opinions with the world. This paper presents a comparative analysis of different deep learning models, including You Only Look Once (YOLO), Convolutional Neural Network (CNN), for translating ASL gestures into English using computer vision techniques. A dataset consisting of images of ASL gestures captured under various lighting conditions was chosen for this study.Existing ASL translation systems often rely on expensive hardware or limited datasets, whereas this study explores an efficient and scalable solution using deep learning.YOLO enables real-time detection, CNN captures spatial features, achieving accuracies of 99%, and 94%, respectively.**

***Index Terms*—ASL, YOLO, CNN, Computer Vision**

I. Introduction

Effective communication is essential for sharing thoughts and information, but it remains a challenge for individuals who are deaf or mute. American Sign Language (ASL) provides a vital means of interaction using hand gestures, facial expressions, and body movements. However a communication barrier often exists when interacting with individuals unfamiliar with ASL. This can be effectively overcome using deep learning and computer vision technologies, which have broad applications ranging from language morphological synthesis [1] to disease identification and classification through medical imaging [2-4].

Advancements in deep learning and computer vision offer promising solutions for ASL translation. Traditional methods often rely on sensor-based gloves or specialized hardware, whereas modern techniques use image and video processing for more accessible and scalable solutions. This study introduces a custom ASL dataset designed to improve gesture recognition across varied conditions and compares two deep learning models—You Only Look Once (YOLO) and Convolutional Neural Networks (CNNs). YOLO enables real-time detection, while CNNs extract spatial features, providing a robust approach to ASL translation.

The relevance of this research lies in its potential to enhance assistive technologies, making communication more inclusive. Possible applications include real-time ASL-to-text conversion for mobile applications, smart devices, and educational tools, improving accessibility for the hearing-impaired community.

Despite its advantages, ASL recognition faces challenges such as variations in hand gestures, changing lighting conditions, occlusions, and the requirement for extensive labeled datasets. Additionally, distinguishing similar gestures and processing continuous signing sequences pose significant difficulties. This study aims to address these challenges by developing a robust dataset and employing deep learning techniques to enhance ASL recognition accuracy, paving the way for practical and real-time ASL translation systems.

II. Related work

Recent advancements in machine learning and computer vision have paved the way for automated sign language recognition and translation systems, offering potential solutions to bridge this gap. There are many proposals that attempt to improve the recognition of ASL through the use of various machine learning models. The conventional approach depends on deep learning methods like CNNs for hand gesture classification, which can be found in the work of ASL Static Gesture Recognition using Deep Learning and Computer Vision [5]. CNNs have been particularly effective in classifying static ASL gestures. An alternative approach explored in American Sign Language Recognition and Translation using Deep Learning and Computer Vision utilizes CNN-based models with image processing techniques to enhance gesture detection accuracy [6].

Beyond deep learning, real-time sign language recognition has been a key focus of research. The Real-Time Sign Language Translation using Computer Vision and Machine Learning study implements Mediapipe for preprocessing ASL

gestures before passing them to machine learning models, making real-time translation more efficient [7]. Similarly, An AI-Based Framework for Translating American Sign Language to English and Vice-Versa introduces a web-based tool powered by TensorFlow and computer vision algorithms to enable sign-to-text conversion, offering a practical and scalable solution [8].

Other than deep learning techniques, image processing algorithms have also been used in ASL translation. In ASL Translation via Image Processing Algorithms, the techniques used for effective segmentation and recognition of sign language gestures include Canny edge detection and hand contour extraction [9]. A Sign Language-to-Text Translator Using Machine Learning and Computer Vision proposed a web-based application called Veritas that utilizes computer vision models to translate ASL gestures into readable text [10].

Recently, You Only Look Once (YOLO) has emerged as an effective real-time object detection model for ASL recognition. Some research has explored its use alongside CNNs and RNNs for ASL-to-English translation. Comparative analysis shows that YOLO can achieve higher real-time accuracy compared to CNNs and RNNs, making it a promising approach for real-world applications [11].

This paper aims to build on these advancements by comparing the performance of YOLO, and CNN models in ASL translation. By leveraging computer vision and deep learning techniques, we seek to develop an efficient and scalable system for translating ASL into English, improving accessibility and inclusivity for the deaf community.

III. Methodology

A. Dataset

The dataset comprises a total of 1,200 images, with 960 images allocated for training and the remaining 240 for testing. The letters J and Z are excluded from the dataset, as their representation in sign language requires motion, which cannot be effectively captured in static images.

B. YOLO

1) Data Preprocessing: The images were captured using OpenCV and NumPy. Following image collection, preprocessing steps were carried out to prepare the data for annotation and training.

2) Annotation: Each image was annotated using the MakeSense.AI tool to make the object detection more easier. This tool helps with the assignment of class labels to objects of interest, producing annotations in a format compatible with the YOLO framework.

3) Model training: The imported dataset was used to train the YOLO model. During training, several parameters needed to be specified, including:

- **Image size**: Define input image size.
- **Batch size**: Determine batch size.
- **Number of epochs**: Define the number of training epochs.
- **Weights**: Specify a custom path to weights.
- **Data**: Set the path to the `.yaml` file.
- **Configuration**: Specify the model configuration.
- **Name**: Result naming convention.
- **Cache**: Cache images for faster training.

These parameters were carefully tuned to optimize model performance.

4) Testing: After training, the YOLO model is used to make predictions on test images. When a test image is provided, the model processes it and identifies the objects present, classifying them based on the trained classes.

This approach ensures that the YOLO model accurately detects and classifies objects in real-world images.

C. CNN

1) Data Preprocessing: All images in the dataset are resized to 64×64 pixels and normalized to the range [0, 1]. The dataset is shuffled to ensure randomness during training. The data is split into:

- **Training set:** Contains all data except the last 10% of the dataset.
- **Validation set:** Contains the last 10% of the dataset from the combined dataset.

2) CNN Architecture: A custom CNN is designed using the Keras Sequential API. The architecture of this CNN consists of:

- Three convolutional layers with increasing filters: $32 \rightarrow 64 \rightarrow 128$.
- Each convolutional layer is followed by BatchNormalization and MaxPooling.
- GlobalAveragePooling is used instead of flattening to reduce overfitting.
- A fully connected dense layer with 512 neurons and ReLU activation introduces non-linearity.
- Dropout with a rate of 0.5 is used for regularization.
- The output layer uses softmax activation for multi-class classification.

3) Regularization Techniques:

- L2 regularization is applied to all convolutional and dense layers.
- Dropout is added before the final output layer to prevent overfitting.

4) Training Details:

- **Optimizer:** Adam optimizer with a learning rate of 0.0005.
- **Loss Function:** Sparse Categorical Crossentropy (as labels are integers).
- **Epochs:** Trained for 30 epochs with a batch size of 32.
- **ModelCheckpoint:** Used to save the best model based on validation accuracy.

5) Testing:

- Loads the testing dataset.
- Uses the saved model to predict the class of each image.
- Compares the predicted values to the true values for evaluation.

IV. Result and Analysis

A. Evaluation metrics

The performance of the YOLO and CNN models was measured using standard detection and classification metrics such as Precision, Recall, F1-Score, Accuracy, and Mean Average Precision (mAP). For a more detailed analysis of YOLO's object detection performance, both mAP@50 and mAP@50:95 were used, providing information about the model's accuracy at various Intersection over Union (IoU) thresholds. These measurements altogether offered a solid assessment platform to contrast the classification power of CNN with the localization and detection effectiveness of YOLO.

B. YOLO

YOLO model performed just about perfect performance, with a precision of about 0.99, 1.0 recall, and an mAP@50 score of 0.995 in all classes as seen in Figure 1. The mAP@50:95 scores scored between 0.88 to 0.93, revealing a minor fall in performance due to more restrictive IoU limits, but with still very high generalization potential. Training graphs showed consistent convergence, with monotonic loss diminishment and the increase in the mAP measure, showing a stable and strong learning process in the course of training.

The training and validation curves of the YOLO model as seen in Figure 2 exhibit stable and robust learning across all 100 epochs. All components of loss—box loss, objectness loss, and classification loss—are monotonically decreasing, reflecting successful optimization of the training process. The validation losses closely follow the training losses, reflecting negligible overfitting and good generalization to new data. Precision and recall scores improve monotonically, both trending towards 1.0, reflecting outstanding detection and classification performance. The mAP@0.5 curve climbs quickly and flattens around 1.0, revealing practically perfect detection with a threshold of 0.5 IoU. Second, the mAP@0.5:0.95 improves further and rests just above 0.95, underlining the power of the model even with higher localization standards. The curves essentially endorse the efficiency and trustworthiness of YOLO's model for live sign language classifying tasks in real-time operations.

Figure 3 illustrates the performance of the YOLO model on ASL gestures from the test set. The 'a', 'e', and 'l' letters are accurately detected in real time, with prominent bounding boxes and high confidence values (e.g., 0.96, 0.95, and 0.97 respectively). The above results illustrate the YOLO model's efficiency in simultaneously localizing and classifying hand signs, asserting its high accuracy and real-time applicability.

Class	Images	Instances	P	R	mAP50	mAP50-95:
all	240	240	0.992	1	0.995	0.904
a	240	10	0.989	1	0.995	0.906
b	240	10	0.992	1	0.995	0.931
c	240	10	0.993	1	0.995	0.836
d	240	10	0.992	1	0.995	0.932
e	240	10	0.993	1	0.995	0.922
f	240	10	0.99	1	0.995	0.939
g	240	10	0.993	1	0.995	0.93
h	240	10	0.991	1	0.995	0.935
i	240	10	0.991	1	0.995	0.936
k	240	10	0.992	1	0.995	0.869
l	240	10	0.989	1	0.995	0.912
m	240	10	0.999	1	0.995	0.948
n	240	10	0.995	1	0.995	0.886
o	240	10	0.989	1	0.995	0.903
p	240	10	0.989	1	0.995	0.938
q	240	10	0.994	1	0.995	0.838
r	240	10	0.993	1	0.995	0.902
s	240	10	0.99	1	0.995	0.845
t	240	10	0.99	1	0.995	0.931
u	240	10	0.994	1	0.995	0.855
v	240	10	0.991	1	0.995	0.912
w	240	10	0.992	1	0.995	0.886
x	240	10	0.991	1	0.995	0.916
y	240	10	0.99	1	0.995	0.888

Fig. 1. Per-class evaluation metrics of the YOLO model including precision (P), recall (R), mAP@0.5, and mAP@0.5:0.95 for each letter.

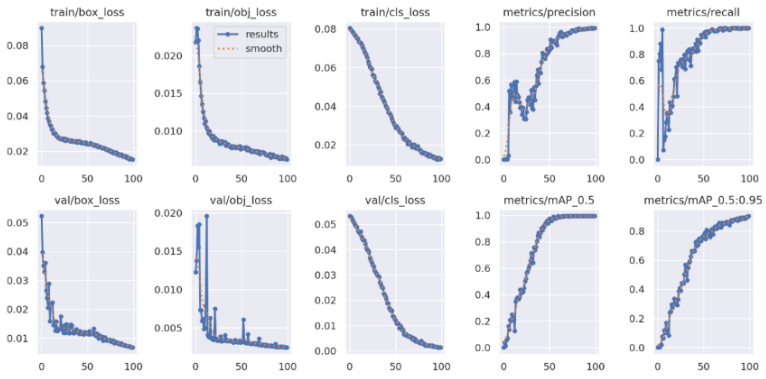

Fig. 2. Training and validation curves of the YOLO model. It shows box, objectness, and classification losses along with evaluation metrics including precision, recall, mAP@0.5, and mAP@0.5:0.95 across 100 epochs.

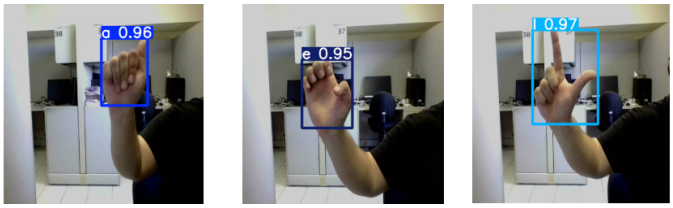

Fig. 3. Live detection results of the YOLO model on ASL gestures: letters 'a', 'e', and 'l' are successfully detected and classified with confidence scores.

C. CNN

The CNN model achieved an overall accuracy of 94% on the test set. Several classes, including 'a', 'b', 'i', 'k', 'p', 'e', 'q', and 'r', recorded high F1-scores close to 1.0, indicating strong classification performance. However, the model's performance declined for certain classes such as 'v', 'u', 'w', 'x', where F1-scores dropped to as low as 0.74 as shown in Figure 4. This difference in class-wise performance indicates the occurrence of class imbalance or visual similarities among some hand gestures, and hence higher rates of misclassification might have been there in those categories.

The plot in Figure 5 indicates the training results of a CNN model, reflecting both accuracy and loss measures in 30 epochs. On the left, the "Accuracy Over Epochs" plot shows how the training accuracy continuously improves and achieves almost flawless performance by roughly the 15th epoch. But the validation accuracy is low at first and only starts to improve noticeably after the 20th epoch, eventually reaching the training accuracy. On the right, the "Loss Over Epochs" graph indicates that the training loss always goes down, reflecting good learning. The validation loss, on the other hand, rises sharply at first and peaks at around the 12th epoch before slowly coming down. This action implies that the model could have suffered from overfitting in the beginning of training but thereafter was able to generalize more effectively, perhaps through adjustments such as learning rate scheduling or techniques of regularization.

Figure 6 displays the CNN model's prediction of ASL gestures on the test set. Each of the images has been marked by the ground truth ("True") and the corresponding output by the model ("Pred"), all correctly matching in each of the presented samples. This further supports the CNN model in generalizing on new data to a good extent, especially in the case of unique gestures like 'f', 'g', 'a', and 'b'. The image is a visual confirmation of the model's accurate test and an endorsement of its efficiency in static ASL image classification tasks.

D. Comparative insights

YOLO performed better than the CNN model on all metrics tested, showing better precision and stable performance across all classes. Although the CNN model was effective in general,

	precision	recall	f1-score	support
a	1.00	0.90	0.95	10
b	1.00	1.00	1.00	10
c	1.00	0.80	0.89	10
d	0.83	1.00	0.91	10
e	1.00	1.00	1.00	10
f	1.00	1.00	1.00	10
fn	0.00	0.00	0.00	0
g	1.00	0.90	0.95	10
h	0.91	1.00	0.95	10
i	1.00	1.00	1.00	10
k	1.00	1.00	1.00	10
l	1.00	0.90	0.95	10
m	0.91	1.00	0.95	10
n	0.91	1.00	0.95	10
o	1.00	0.90	0.95	10
p	1.00	1.00	1.00	10
q	1.00	1.00	1.00	10
r	1.00	0.90	0.95	10
s	1.00	1.00	1.00	10
sp	0.00	0.00	0.00	0
t	0.91	1.00	0.95	10
u	0.59	1.00	0.74	10
v	0.90	0.90	0.90	10
w	1.00	0.70	0.82	10
x	1.00	0.70	0.82	10
y	1.00	1.00	1.00	10
accuracy			0.94	240
macro avg	0.88	0.87	0.87	240
weighted avg	0.96	0.94	0.94	240

Fig. 4. Classification report of the CNN model showing precision, recall, f1-score, and support for each letter. The model achieves high overall accuracy, with some variability in individual class performance.

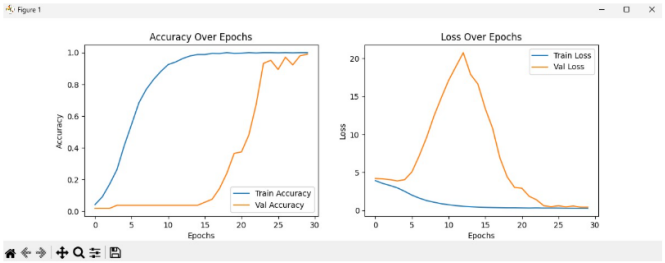

Fig. 5. Training and validation curves of the CNN model. The left plot shows accuracy over epochs, and the right plot shows loss. The validation performance improves after epoch 20, indicating effective learning.

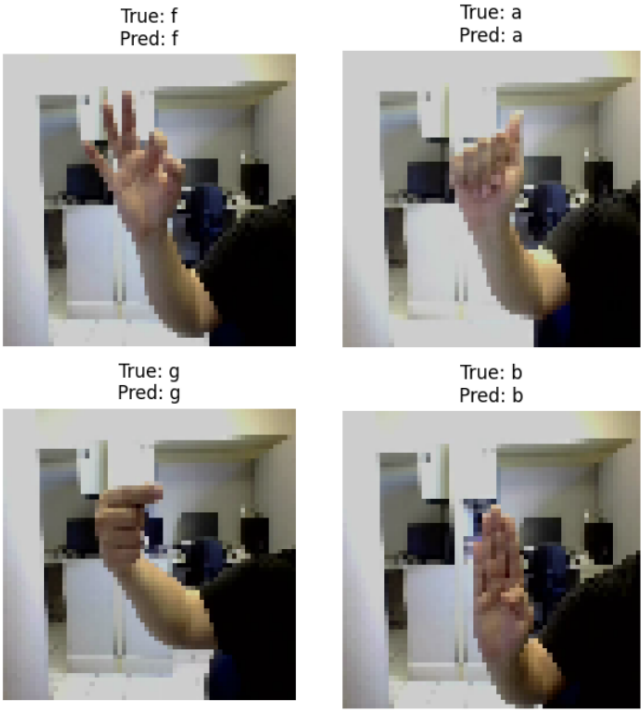

Fig. 6. Predictions from the CNN model on test ASL gesture images. The true and predicted labels are displayed above each image. The model correctly classifies all the shown samples.

it showed variability in class-wise performance, indicating the necessity for further enhancement using data augmentation or architectural optimization. With its high accuracy, stability across various classes, and fast object detection capability, YOLO is a more robust and appropriate solution for real-time applications.

E. Discussion on YOLO's Superior Performance

One of the most significant reasons why YOLO was superior to the CNN model is due to its object detection-based structure, which automatically localizes and classifies individual hand signs in the space of images. Such spatial cognition enables YOLO to effectively distinguish between small gesture differences even under cluttered or visually analogous conditions. Conversely, the CNN model processes the entire image as a single input without localizing regions of interest, which may cause ambiguity in classes with fine-grained distinctions. Moreover, YOLO takes advantage of data augmentation, anchor-based bounding boxes, and multi-scale feature detection, so it is more invariant to hand shape, size, and position variations. These factors lend themselves to its improved consistency, particularly in cases when the CNN would be hindered by overfitting or by a lack of region-based understanding.

V. Conclusion and Future Work

This paper introduced a comparative study between a YOLO-based object detection model and a custom Convolutional Neural Network (CNN) for static American Sign Language (ASL) alphabet classification. Both methods showed encouraging results, with the CNN recording a high test accuracy of 94%, and YOLO exhibiting strong real-time detection performance with high per-class precision and recall scores. The outcomes showed that most ASL letters were correctly classified, but some visually similar signs—specifically those with small finger differences—were difficult for both models, particularly in less ideal or more variable lighting.

There are a number of opportunities to enhance the system as it stands. Misclassification of letters 'u', 'v', and 'w' indicates the necessity of improved feature extraction, perhaps by incorporating hand keypoint detection or depth sensors. In addition, increasing the training data set with more varied samples—such as differences in hand shape, complexion, and background—would potentially enhance generalization. Future studies might also address dealing with dynamic gestures like 'j' and 'z' with sequence models like 3D CNN or recurrent neural networks, which would make it possible for a broader recognition system.

In order to enable real-time deployment, the upcoming deployments can take advantage of light deep learning libraries and edge device hardware acceleration such as NVIDIA Jetson Nano or Raspberry Pi. The platform can further be expanded to practical applications like sign language interpretation, hearing-impaired accessibility tools, and education platforms. In addition, combining this technology with speech recognition or text-to-speech systems would open the door to a full ASL-to-speech pipeline, leading to more accessible human-computer interaction and communication.

References

[1] B. Premjith and K. P. Soman, "Deep learning approach for the morphological synthesis in Malayalam and Tamil at the character level," Trans. Asian Low-Resour. Lang. Inf. Process., vol. 20, no. 6, pp. 1–17, 2021.

[2] K. Jaswanth, M. T. Nikhil, M. S. S. Siddhartha, and S. Jayan, "Comparative analysis of COVID detection using chest X-rays by SVM-PCA and deep learning techniques," in Proc. 2023 Int. Conf. Adv. Electron., Commun., Comput. Intell. Inf. Syst. (ICAECIS), Apr. 2023, pp. 188–193.

[3] A. Amruth, R. Ramanan, R. Paul, S. Jayan, A. Thakur, and N. P. Tv, "Comparative Study of Cancer Classification by Analysis of RNA-seq Gene Expression Levels," in Proc. 2022 13th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT), Oct. 2022, pp. 1–7.

[4] C. S. Sresti, S. H. Sai, V. B. Sharma, and S. Jayan, "Skin Disease Classification using VGG-16 model optimized ADAM, GD, RMSprop and YOLOv8," in Proc. 2024 1st Int. Conf. Innov. Commun., Electr. Comput. Eng. (ICICEC), Oct. 2024, pp. 1–7.

[5] P. M. Patel, M. M. Patel, and S. J. Patel, "ASL Static Gesture Recognition using Deep Learning and Computer Vision," in *Proc. Int. Conf. Comput. Intell. Comput. Appl. (ICCICA)*, 2021.

[6] S. Chandrasekaran, "American Sign Language Recognition and Translation using Deep Learning and Computer Vision," M.Sc. thesis, National College of Ireland, 2021. [Online]. Available: https://norma.ncirl.ie/5140/1/sruthichandrasekaran.pdf

[7] A. Sharma, A. Tyagi, A. Tomar, A. Srivastava, and A. Srivastava, "Real-Time Sign Language Translation using Computer Vision and Machine Learning," in *Proc. Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, 2024, pp. 10–14. doi: 10.1109/ICICCS2024.10533962.

[8] K. Bhattacharya et al., “An AI-Based Framework for Translating American Sign Language to English and Vice-Versa,” *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/381344315

[9] M. A. Wahid, M. S. Islam, and M. A. Rahman, “American Sign Language Translation via Image Processing Algorithms,” in *Proc. Int. Conf. Innov. Sci., Eng. Technol. (ICISET)*, 2016, pp. 1–5. doi: 10.1109/ICISET.2016.7519386.

[10] A. Tiwari and D. Vijay, “A Sign Language to Text Translator Using Machine Learning and Computer Vision,” in *Proc. ACM Conf. Comput. Sustain. Soc. (COMPASS)*, 2021. doi: 10.1145/3507623.3507633.

[11] [Anonymous], “American Sign Language (ASL) to English Translation Using YOLO, CNN, and RNN,” Unpublished Manuscript. Abstract referenced in current paper.